

HDOC 30

Name: Keshav Mittal SAP ID: 590032934

Batch: 16

1. Algorithm

1. Start
2. Read array size and elements. Validate size.
3. Display the menu (1. Sort & Search, 2. Insert/Delete, 3. Rotate, 4. Exit) and read choice.
4. Case 1 (Sort & Search): Apply Bubble sort. Display sorted array. Read key. Perform Binary Search by halving the search interval until the key is found or interval is empty. Display result.
5. Case 2 (Insert/Delete): Read sub-choice.
 - If Insert: Read position and element. Shift elements from end down to position. Insert element. Increment size.
 - If Delete: Read position. Shift elements from position to end left by one. Decrement size.
6. Case 3 (Rotate Right): Read k. Calculate $k = k \% \text{size}$. Store last k elements in a temporary array. Shift remaining elements right by k. Copy temporary elements to the beginning.
7. Repeat until choice is 4.
8. Stop

2. Test-Case Table

TC ID	Scenario	Inputs	Expected Output
TC-01	Bubble Sort & Binary Search	Arr: 5 2 9 1, choice: 1, key: 5	Sorted: 1 2 5 9. Found at index 2.
TC-02	Insert element	Arr: 1 2, choice: 2, sub: 1, pos: 1, val: 9	Array: 1 9 2
TC-03	Delete element	Arr: 1 9 2, choice: 2, sub: 2, pos: 1	Array: 1 2
TC-04	Rotate right	Arr: 1 2 3 4, choice: 3, k: 1	Array: 4 1 2 3
TC-05	Invalid option	choice: 5	Error message displayed.

3. C Program Source Code

```
#include <stdio.h>

void displayArray(int arr[], int size) { for (int i = 0; i < size;
    i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void bubbleSort(int arr[], int size) { int temp;
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) { if (arr[j] > arr[j + 1]) {
            temp = arr[j]; arr[j] = arr[j +
            1]; arr[j + 1] = temp;
        }
    }
}

int binarySearch(int arr[], int size, int key) { int low = 0, high = size - 1,
    mid;
    while (low <= high) {
        mid = low + (high - low) / 2; if (arr[mid] ==
        key)
            return mid;
        if (arr[mid] < key) low = mid
            + 1;
        else
            high = mid - 1;
    }
    return -1;
}

void insertElement(int arr[], int *size, int pos, int element) { if (pos < 0 || pos > *size) {
    printf("Invalid position!\n"); return;
}
    for (int i = *size; i > pos; i--) { arr[i] = arr[i - 1];
    }
    arr[pos] = element; (*size)++;
}

void deleteElement(int arr[], int *size, int pos) { if (pos < 0 || pos >= *size) {
    printf("Invalid position!\n"); return;
}
    for (int i = pos; i < *size - 1; i++) {
```

```

        arr[i] = arr[i + 1];
    }
    (*size)--;
}

void rightRotate(int arr[], int size, int k) { if (size == 0) return;
    k = k % size;
    if (k < 0) k += size;

    int temp[k];
    for (int i = 0; i < k; i++) { temp[i] = arr[size - k + i];
    }
    for (int i = size - 1; i >= k; i--) {
        arr[i] = arr[i - k];
    }
    for (int i = 0; i < k; i++) { arr[i] = temp[i];
    }
}

int main() {
    int arr[100];
    int size, choice, subChoice, key, pos, element, k, index;

    printf("Enter the size of the array: "); scanf("%d", &size);

    if (size <= 0 || size > 100) { printf("Invalid array
        size.\n"); return 1;
    }

    printf("Enter %d elements: ", size); for (int i = 0; i <
size; i++) {
        scanf("%d", &arr[i]);
    }

    do {
        printf("\n--- Array Operations Menu ---\n"); printf("1. Sort (Bubble Sort) and
        Binary Search\n"); printf("2. Insertion and Deletion\n");
        printf("3. Rotate right by k positions\n"); printf("4. Exit\n");
        printf("Enter your choice: "); scanf("%d",
        &choice);

        switch(choice) { case 1:
            bubbleSort(arr, size); printf("Sorted Array: ");
            displayArray(arr, size);

            printf("Enter element to search: "); scanf("%d",
            &key);
            index = binarySearch(arr, size, key);

```

```

        if (index != -1) {
            printf("Element %d found at index %d.\n", key, index);
        } else {
            printf("Element %d not found in the array.\n", key);
        }
        break;
}

case 2:
    printf(" 1. Insert an element\n"); printf(" 2. Delete
    an element\n"); printf(" Enter choice: "); scanf("%d",
    &subChoice);

    if (subChoice == 1) {
        printf("Enter position (0 to %d): ", size); scanf("%d", &pos);
        printf("Enter element to insert: "); scanf("%d", &element);

        insertElement(arr, &size, pos, element); printf("Array after
        insertion: "); displayArray(arr, size);

    } else if (subChoice == 2) {
        printf("Enter position to delete (0 to %d): ", size - 1); scanf("%d", &pos);

        deleteElement(arr, &size, pos); printf("Array after deletion: ");
        displayArray(arr, size);

    } else {
        printf("Invalid option.\n");
    }
    break;

case 3:
    printf("Enter number of positions to rotate right: "); scanf("%d", &k);
    rightRotate(arr, size, k); printf("Array after
    rotation: "); displayArray(arr, size);
    break;

case 4:
    printf("Exiting program...\n"); break;

default:
    printf("Error: Invalid choice. Please enter 1-4.\n");
}
} while (choice != 4);

return 0;
}

```

5. Compilation and Execution

```
(kali@kali)-[~/liak]
$ ./a.out
Enter the size of the array: 4
Enter 4 elements: 653 543 8765 2

— Array Operations Menu —
1. Sort (Bubble Sort) and Binary Search
2. Insertion and Deletion
3. Rotate right by k positions
4. Exit
Enter your choice: 1
Sorted Array: 2 543 653 8765
Enter element to search: 2
Element 2 found at index 0.

— Array Operations Menu —
1. Sort (Bubble Sort) and Binary Search
2. Insertion and Deletion
3. Rotate right by k positions
4. Exit
Enter your choice: 2
  1. Insert an element
  2. Delete an element
  Enter choice: 2
Enter position to delete (0 to 3): 4
Invalid position!
Array after deletion: 2 543 653 8765

— Array Operations Menu —
1. Sort (Bubble Sort) and Binary Search
2. Insertion and Deletion
3. Rotate right by k positions
4. Exit
Enter your choice: 4
Exiting program...
```